COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	09/11/2026	**DATE SUBMITTED:**	09/16/2026
NAME:	Moradillo, Eunice	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 8.1 Using Sorting Algorithms

1. PROCEDURE OUTPUT

Code + Console Screenshot

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingAlgo.h"

const int maxSize = 100;
void displayArr(int arr[]);


int main(){

    //create an array with unsorted list
    int unsortedArr[maxSize];
    //use time (0) as seed for random values
    srand(time(0));
    //populate the array with random values
    for(int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand() % 500; //random values between 0 and 499
    }
    //display the randomly generated array
    std::cout<<"unsorted array: "<<std::endl;
    displayArr(unsortedArr);
```

```
unsorted array:
220 53 387 9 342 207 307 285 426 118 171 97 339 101 301 44 247 211 453 30 104 440 87 248 325 263 173
302 36 197 146 461 102 183 130 451 124 254 87 345 205 286 483 309 307 12 459 147 287 303 339 341 359
229 347 293 71 383 89 446 439 410 376 335 454 252 81 295 328 178 160 270 207 107 145 123 440 390 299
378 237 307 33 383 94 91 23 464 276 116 28 308 103 131 149 157 307 89 144 287
```

Observations

Similarly to the last Hands On Activity, we first generated a hundred random values from 0 to 500 and in an unsorted array srand(time(0)). This unsorted array served as the common input for all three sorting algorithms tested in the activity.

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot

```
std::cout<<"shell sort: "<<std::endl;
shellSort(unsortedArr, maxSize);
displayArr(unsortedArr);
```

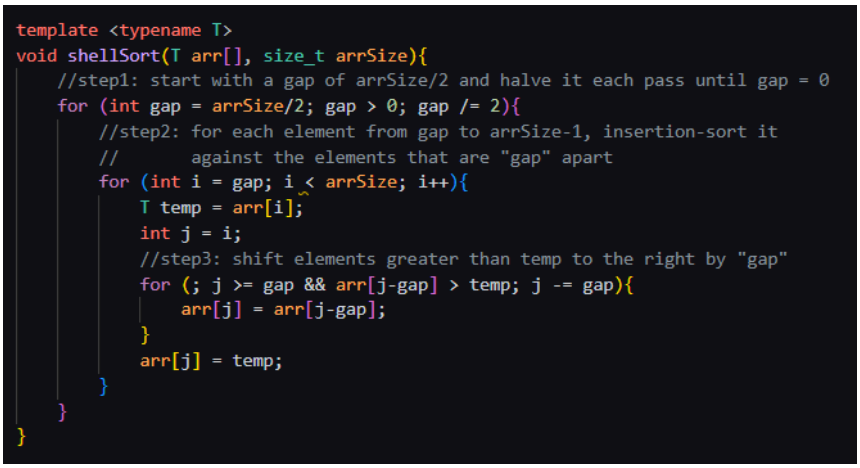
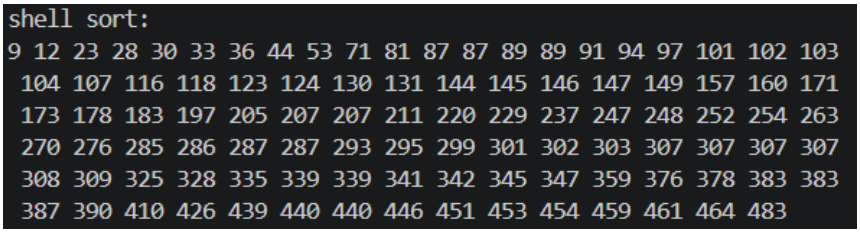
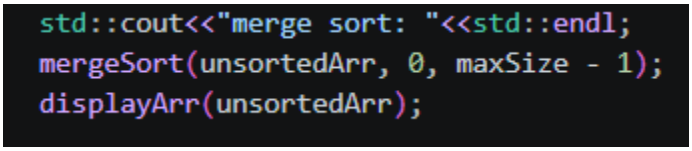
	 
Observations	In this code, we start with a gap of half the array size and use it to insertion-sort elements that many positions apart, halving the gap each pass until it reaches zero. The array of 100 values is sorted correctly in ascending order from 9 up to 483.

Table 8-2. Shell Sort Technique

Code + Console Screenshot	

```
template <typename T>
void mergeSort(T arr[], int left, int right){
    //step1: base case, a sub list of 0 or 1 elements is already sorted
    if (left < right){
        int mid = left + (right - left)/2;
        //step2: sort the left half and right half recursively
        mergeSort(arr, left, mid);
        mergeSort(arr, mid+1, right);

        //step3: merge the two sorted halves back together
        int leftSize = mid - left + 1;
        int rightSize = right - mid;

        T* leftArr = new T[leftSize];
        T* rightArr = new T[rightSize];

        for (int i = 0; i < leftSize; i++) leftArr[i] = arr[left+i];
        for (int j = 0; j < rightSize; j++) rightArr[j] = arr[mid+1+j];

        int i = 0, j = 0, K = left;
        //repeatedly place the smaller of the two front elements back into arr
        for (; i < leftSize && j < rightSize; K++){
            if (leftArr[i] <= rightArr[j]){
                arr[K] = leftArr[i];
                i++;
            } else {
                arr[K] = rightArr[j];
                j++;
            }
        }
        //copy over any leftover elements
        for (; i < leftSize; i++, K++) arr[K] = leftArr[i];
        for (; j < rightSize; j++, K++) arr[K] = rightArr[j];

        delete[] leftArr;
        delete[] rightArr;
    }
}
```

```
merge sort:
9 12 23 28 30 33 36 44 53 71 81 87 87 89 89 91 94 97 101 102 103
 104 107 116 118 123 124 130 131 144 145 146 147 149 157 160 171
 173 178 183 197 205 207 207 211 220 229 237 247 248 252 254 263
 270 276 285 286 287 287 293 295 299 301 302 303 307 307 307 307
 308 309 325 328 335 339 339 341 342 345 347 359 376 378 383 383
 387 390 410 426 439 440 440 446 451 453 454 459 461 464 483
```

Observations	In this code, the array is repeatedly split in half until each part has only one element, then the halves are merged back together in sorted order. The final array matches the same sorted sequence produced by shell sort, from 9 up to 483, confirming both algorithms sorted the same input correctly.

Table 8-3. Merge Sort Algorithm

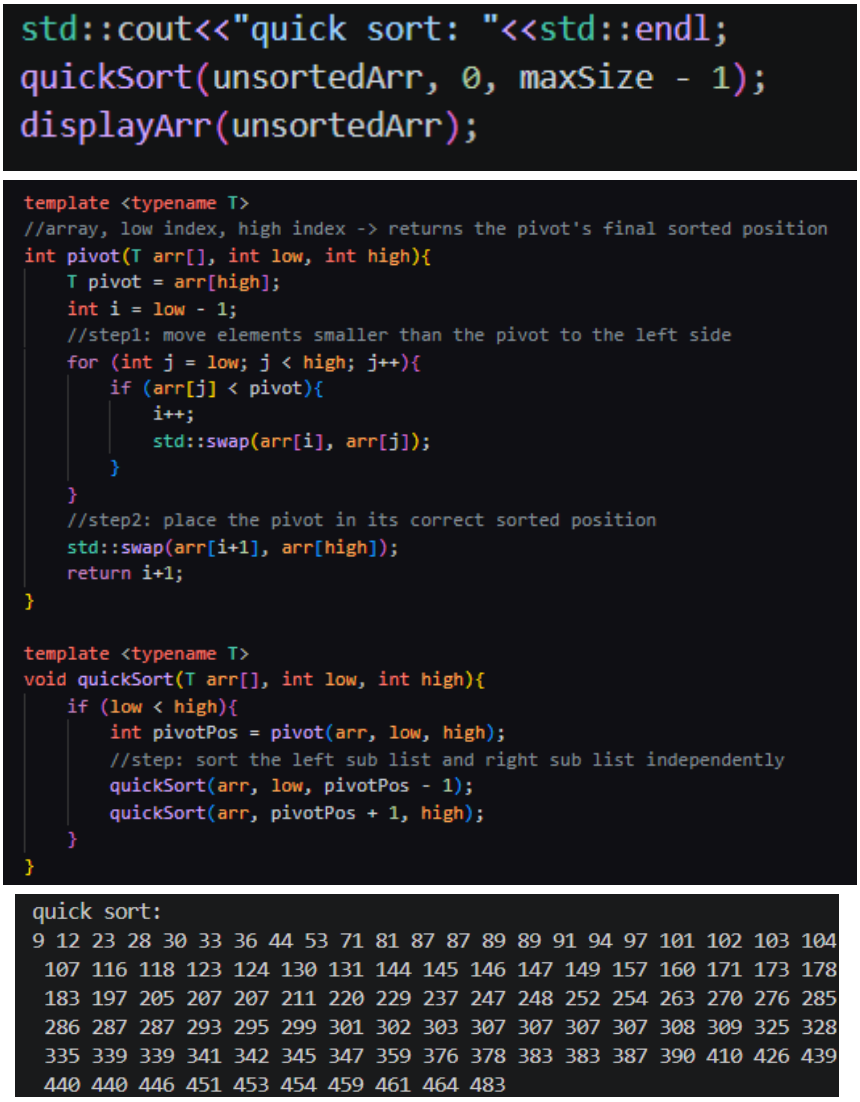
Code + Console Screenshot	
Observations	In this code, a pivot is chosen and the array is partitioned so smaller elements go to its left and larger ones to its right, then the two sides are sorted recursively using the same method. The output again matches the sorted sequence from 9 to 483, showing that all three algorithms produced identical results from the same unsorted array

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

PROBLEM 1

Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

Yes, the left and right sub lists from quicksort's partition method can be sorted using other sorting algorithms. Once the partition step runs, it places the pivot in its correct final sorted position, with every element smaller than the pivot to its left and every element larger to its right. At that point, the two sub lists are just independent, unsorted groups that don't require quicksort specifically to finish sorting them. Using the same example array from the procedure, 10, 42, 15, 12, 37, 7, 9, with the last element (9) as pivot, a single partition step produces 7, 9, 15, 12, 37, 10, 42, placing the pivot at index 1 and splitting the array into a left sub list {7} (already sorted, being a single element) and a right sub list {15, 12, 37, 10, 42}. Instead of recursing back into quicksort, the right sub list can be sorted with insertion sort, which sorts it into 10, 12, 15, 37, 42, giving the final fully sorted array 7, 9, 10, 12, 15, 37, 42, with quicksort's partition only ever called once.

PROBLEM 2

supplementary.cpp > ...

```cpp
#include <iostream>
#include "sortingAlgo.h"

const int maxSize = 15;

void displayArr(int arr[]);

int main(){

    //create an array with unsorted list
    int unsortedArr[maxSize] ={4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19,74};

    std::cout<< "Merge Sort\n";

    std::cout << "Unsorted Array:\n";
    //display the randomly generated arr
    displayArr(unsortedArr);


    std::cout << "\nSorted Array :\n";
    displayArr(unsortedArr);

    std::cout<<std::endl;

```

```
    //reset the array back to its original unsorted order before testing
    //quick sort, since unsortedArr is already sorted from merge sort above
    int unsortedArr2[maxSize] ={4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19,74};

    std::cout<< "\nQuick Sort";

    std::cout << "\nUnsorted Array:\n";
    displayArr(unsortedArr2);


    std::cout << "\nSorted Array :\n";
    displayArr(unsortedArr2);

    return 0;
}

void displayArr(int arr[]){
    for(int i = 0; i < maxSize; i++){
        std::cout << arr[i] << " ";
        if ((i%10) == 9){
            std::cout << '\n';
        }
    }
}
```

```
Merge Sort
Unsorted Array:
4 34 29 48 53 87 12 30 44 25
93 67 43 19 74
Sorted Array :
4 34 29 48 53 87 12 30 44 25
93 67 43 19 74

Quick Sort
Unsorted Array:
4 34 29 48 53 87 12 30 44 25
93 67 43 19 74
Sorted Array :
4 34 29 48 53 87 12 30 44 25
93 67 43 19 74
```

EXPLANATION:
With only 15 elements, both merge sort and quick sort would run in a fraction of a microsecond, so the difference is barely noticeable. Quick sort would likely be slightly faster here since it sorts in place with no extra memory allocation, while merge sort carries some overhead from creating temporary arrays at every merge

step. Merge sort gets O(N log N) because it always splits the array exactly in half, giving log N levels, and merging each level takes O(N) work, so the total is N times log N. Quick sort gets O(N log N) on average for the same reason, since a good pivot splits the array roughly in half each time, but it can drop to $O(N^2)$ if the pivot keeps landing at one end of the array, like with an already-sorted list, causing uneven splits.

CONCLUSION

Through this activity, we implemented shell sort, merge sort, and quick sort in C++ and tested them on the same set of unsorted values to compare how each one works. We learned that shell sort improves on basic insertion sort by comparing elements that are far apart before narrowing the gap, merge sort reliably guarantees O(N log N) performance by always splitting the array evenly, and quick sort achieves similar average performance through partitioning but depends on the pivot choice to avoid its worst case. Completing the supplementary problems also helped us understand that a sorting algorithm's sub-problems don't have to be solved by the same algorithm, and that time complexity differences between algorithms become more apparent as the size of the data set grows. I think I understood the procedure and the concepts behind each algorithm well, though comparing their actual execution times on larger datasets would help my understanding of their performance differences.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.